STEP 2: The View

Now that we have defined what to do with our URLs, we need to handle them when they come to the view. The view has to pull in data from the database based on the part of the site being viewed, and display them to the end user. However, how this data is formatted is defined by a template. Open your views.py file in the books folder; it should be empty, but we'll soon fix that.

The Django template book page

Let us start with just this bit of code:

```
def book(request, book_id):
 return HttpResponse("The id of the book is %s." % book_id)
```

Now go to 127.0.0.1:8000/book/11 and you will see the id of the book on the page. This, as you'd agree, is useless. So let's make it do something. We will make the index page list all the books.

As you can imagine, this is quite cumbersome, especially as sites become more complex. Let's see how templates make this better. First we will create the views that use templates; replace the views.py contents with the following:

```
from django.shortcuts import render_to_response
from models import *

def index(request):
    book_list = Book.objects.all()
    persons_list = Person.objects.all()
    return render_to_response('books/index.html',{
        'books':book_list,
        'persons':persons_list
        })

def book(request, book_id):
    book = Book.objects.get(pk=book_id)
    return render_to_response('books/book.html',{
        'book':book
        })
```

Here is how this works:

- The function render_to_response is a shortcut that loads the template (here 'books/index.html') and passes all the parameters after that. Here, it will pass along the list of books and persons as 'books' and 'persons', respectively. This data will be used by the 'index.html' template.
- Similarly for the book function, the Book.objects.get function retrieves a single book based on the supplied parameter. Here the parameter is 'pk' or the primary key, which it matches against the book_id from the URL. The matched object is passed to the 'book.html' template as 'book'.
- A directory called 'templates' must be created under your project directory, and the 'index.html' and 'book.html' files will be placed in the 'books' subdirectory inside that folder.

STEP 3: The templates

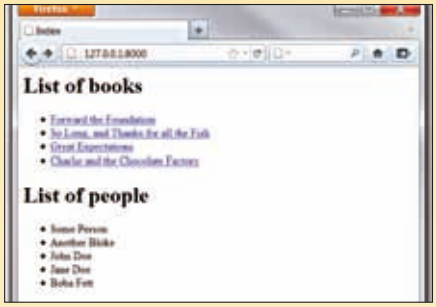
Powerful as they may be, Django templates don't use Python. The template language Django uses is not a programming language, but a presentation language. It can access variable and loop through arrays, but it can't call functions in your code, so design your application likewise.

Let us take a look at how we might make out "index.html" page; we are omitting basic HTML structure here, but feel free to add it:

```
<h1>List of books</h1>
<ul>
{% for book in books %}
<li>{{ book.name }}</li>
{% endfor %}
```

This will show a nice header "List of books" followed by the actual list. Here is what's going on:

- Django tags expressed as {% tag %}. Here we use a "for" tag to iterate over the list of books. This syntax might seem familiar to those who have used other programming languages.
- The "for" tag will repeat the contents between it and the "endfor" tag on each iteration. Here it will repeat for each book in the database
- You can print variables by using this syntax: {{ variable }}. Here we print the name of the book as {{ book.name }}.
- You can apply filters to variables. For example {{ book.name|lower }} will output the name of the book converted to lower case. There are many useful filters; another example {{ size|filesizeformat }} will convert the number stored in 'size' to a filesize representation i.e. in KB, MB etc.



The Django template index

Let's make this a little better and actually have each book link to its page. Replace contents of the for loop with: `<li><a href="/book/{{ book.pk }}">{{ book.name }}</a></li>`

You can as easily create a list of all persons too by adding:

```
<h1>List of people</h1>
<ul>
{% for person in persons %}
<li>{{ person.first_name }} {{ person.last_name }}</li>
{% endfor %}
</ul>
```

Creating the 'book.html' file is as simple:

```
<h1>{{ book.name }}</h1>
{% if book.borrowed_by %}
<p>Borrowed by: {{ book.borrowed_by }}</p>
{% endif %}
{% if book.borrowed_on %}
<p>Borrowed on: {{ book.borrowed_on }}</p>
{% endif %}
```

Hopefully this will need little explanation. It displays the book name as the heading, the name of the person who borrowed it and the date on which it was borrowed. The latter part will only be displayed if they are set in the first place.